

Contextual Agent Security: A Policy for Every Purpose

Lillian Tsai
Google
tslilyai@google.com

Eugene Bagdasarian
Google
ebagdasa@google.com

Abstract

Judging an action’s safety requires knowledge of the *context* in which the action takes place. To human agents who act in various contexts, this may seem obvious: performing an action such as email deletion may or may not be appropriate depending on the email’s content, the goal (e.g., erase sensitive emails or clean up trash), and the type of email address (e.g., work or personal). Unlike people, computational systems have often had only limited agency in limited contexts. Thus, manually crafted policies and user confirmation such as smartphone app permissions or access control lists—while imperfect—have sufficed to restrict harmful actions. However, with the upcoming deployment of generalist agents that support a multitude of tasks (e.g., an automated personal assistant), we argue that we must rethink security designs to adapt to the scale of contexts and capabilities of these systems. As a first step, this paper explores contextual security in the domain of agents and proposes *contextual agent security* (Conseca), a framework to generate just-in-time, contextual, and human-verifiable security policies.

CCS Concepts

• Security and privacy → Operating systems security; Vulnerability management; Access control; • Computing methodologies → Artificial intelligence.

ACM Reference Format:

Lillian Tsai and Eugene Bagdasarian. 2025. Contextual Agent Security: A Policy for Every Purpose. In *Workshop on Hot Topics in Operating Systems (HOTOS ’25)*, May 14–16, 2025, Banff, AB, Canada. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3713082.3730378>

1 Introduction

Context forms the basis for every action, whether taken by a human or a system: we can disambiguate an action’s meaning only via the context in which it exists. For example, in the context of an urgent deadline, scheduling over a lunch break might be appropriate, while the same action would be

inappropriate for a casual sync. Researchers have already integrated context to understand privacy in computer systems [5, 7, 9, 11, 15, 30, 39, 48, 51, 52, 66, 74], which also manifests as appropriateness in AI agents [41]. Importantly, in adversarial settings, analyzing surrounding context is necessary to differentiate harmful from benign actions.

However, approaches today for systems security depend on manually-specified contexts, or no context at all. Conventional security systems often use policy enforcers and access controls [1, 6, 12–14, 18, 29, 45, 46, 49, 56, 64, 67, 83, 86]. While these can support fine-grained policies and capabilities, their rules are (manually) prewritten and policies static. Because policy or rule writers cannot account for every possible context, these rules can overrestrict actions in some contexts, overallow actions in others, and be challenging to write [37, 63]. Some systems’ reliance on e.g., manual user confirmation can also lead to user fatigue and overpermissioning [19, 21, 24, 33, 57, 61]. These limitations, while affecting any multi-purpose system like a filesystem or OS, have become increasingly important to address with the rise of systems like generalist (AI) agents [3, 25, 62, 79, 87], which may serve a far broader range of purposes and thus encounter far broader contexts.

We make the key observation that as the scale of contexts encountered by a system increases, so must the granularity of policies; otherwise, potential over- or under-permissioning may significantly impair utility or security. To this end, we need more research into how to secure systems with broad capabilities and purposes.

As a first step to explore how to scale security to match growing system capabilities, we focus on the domain of agents (AI and otherwise) and propose *contextual agent security* (Conseca¹), a framework that moves us closer toward a world where every action has a contextually-appropriate, purpose-driven justification. Conseca demonstrates a possible way to integrate context at scale to achieve both better utility and better security than static security policies: Conseca generates just-in-time, contextual, and transparent security policies that specify which actions are not harmful for a given context and purpose (and restrict all other actions). Policy enforcement is deterministic, making enforcement impervious to attacks like prompt injections [32, 54, 75].



This work is licensed under a Creative Commons Attribution 4.0 International License.

HOTOS ’25, Banff, AB, Canada

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1475-7/2025/05

<https://doi.org/10.1145/3713082.3730378>

¹“Conseca” is the second-person singular present active imperative of the Latin verb *conseco*. *Conseco* can mean e.g., “to cut” or “to prune”.

Scaling security for systems with many purposes and contexts faces two main challenges. First, Conseca must produce policies at per-context scale. Ideally, personal security experts would manually craft a custom policy for each encountered context, but this is impractical to realize. To solve this scalability challenge, Conseca uses the next-best proxy—a language model that can be adapted to understand user norms and security—to dynamically produce security policies for fine-grained contexts. To help mitigate errors from the policy language model, Conseca’s policies include human-readable rationales that can be audited by experts.

Second, policy generation must be robust to adversarial manipulation, which is particularly concerning when policy generation uses models. Conseca prevents adversary-controlled context (e.g., a sent email) from altering policies by isolating the policy model to handle only trusted context (e.g., user-provided data). Ideally, a model would have full context to best define appropriate actions; trusted context lets the model predict appropriate actions better than without any context, and more securely than with full context.

This paper contributes the following:

- (1) A call to action for work on security mechanisms that can scale to potentially unknown tasks and contexts, particularly for general-purpose agent systems;
- (2) Conseca, a contextually-aware agent security framework that scales security for generalist agents using generated but deterministically-enforced policies specific to a particular purpose and trusted context; and
- (3) A proof-of-concept prototype of Conseca integrated with a Linux computer-use agent (§4, §5).

2 Background and Threat Model

We focus this paper on **agents**: systems that may fulfill potentially arbitrary user tasks and thus represent the full scope of the challenge to scale security. Given a user task request, an agent determines which actions to take to complete that task, and executes those actions, potentially using external tools [25, 62]. As shown in Figure 1, Conseca abstracts agents into a “planner” component that processes user requests and outputs one or more actions to execute (e.g., bash commands). The “executor” component actually runs actions and interfaces with external tools. The executor returns outputs back to the planner, which may continue proposing actions. Various implementations of agents exist: the planner may be e.g., an LLM (making the agent an **AI agent**) or a deterministic program; the executor can be a separate program, or just lines of code in a planner program. This paper explores agents for **computer use** in particular, an agent application with many capabilities and potential for abuse [3, 53].

The **context** of a user’s request contains all information (including the request) relevant for their requested task. For example, a user may wish to backup their important files:

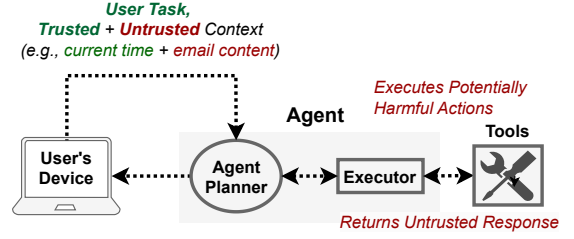


Figure 1: An agent contains a planner and an executor that interfaces with external tools. Untrusted context from the initial request or tool responses may compromise the agent.

the context could include their username; filenames and file contents; job description; IP rights (work or personal); and where they store back-ups. Agents can receive context both directly with a user’s request, or from external tool responses.

2.1 Threat Model

An unrestricted agent (Figure 1) might perform harmful actions, both by mistake, and due to attacks where the **adversary’s goal** is to execute malicious actions via modifying the context (e.g., prompt-injection attacks on models [28, 32, 54]). For example, an attacker could send the user a phishing email which contains a script, and the agent might later be tricked into executing the script when reading the email contents for a `summarize_emails` task.

Conseca addresses these threats in a model where the adversary can only affect the agent’s actions by changing part of the context (e.g., sending a malicious email), and the agent is benevolent (but might make mistakes). As shown in Figure 1, we trust some context (e.g., timestamps), and consider other context untrusted (e.g., third-party emails).

The **agent’s goal** is to execute actions that align with user expectations and work toward fulfilling the user’s request, given the current context (e.g., the emails currently in the user’s inbox). This may require untrusted context (e.g., email contents in order to summarize them).

3 Design

The Conseca framework performs two functions: (i) it takes task requests from the user along with trusted context (§3.1) and generates a task- and context-specific security policy (§3.2); and (ii) it provides feedback to the agent on whether a proposed action satisfies the policy (§3.3).

To use Conseca, the developer installs hooks that invoke (i) the policy generator with trusted context and external tool documentation upon a new user request, and (ii) the policy evaluator to check proposed agent actions. Figure 2 demonstrates how Conseca operates:

(1) Under some context, a user sends a task request to the agent (1a). Conseca’s policy generator processes the request,

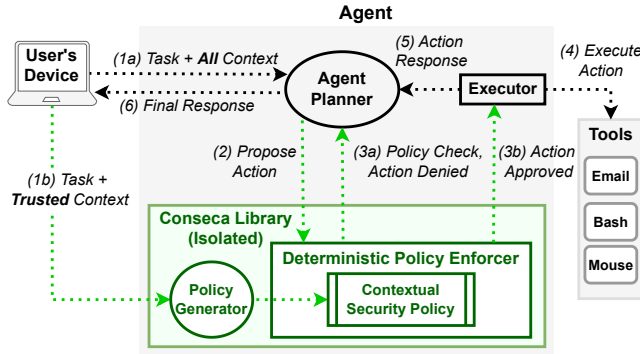


Figure 2: Consecas enables policy generation and enforcement for an example computer use agent with access to external tools. Green lines indicate Consecas’s control flows.

trusted context (1b, §3.1), and tool API documentation, and produces a task- and context-specific security policy (§3.2).

(2-3) The planner proposes the next action to execute for the task, invoking the Consecas policy enforcer. The enforcer (deterministically) evaluates the action: if the policy denies the action, Consecas returns a rationale back to the planner (3a), which can propose a new action or request user confirmation (§7). Otherwise, Consecas forwards the action to execute (3b).

(4-6) The executor invokes a tool (if any) to carry out the action (e.g., sending bash commands to the bash tool); the output returned to the planner may contain untrusted content (e.g., email or file contents). The planner returns to (2) to select the next action to execute based on the task, context, and actions/outputs history. If the agent has completed the user’s task, the user receives the final output.

3.1 Trusted Context

While context is essential to judge an action’s appropriateness, adversaries can manipulate some of the context and thus also influence the policy generator model. Consecas relies on developers to specify what context to trust (typically context not susceptible to potential attackers); alternatively, Consecas could ask users to set coarse-grained trusted context boundaries. Trusting more context can allow Consecas to write a more accurate policy, but also increases the potential for compromise. For example, an email address could in principle encode malicious instructions (address formats typically allow long sequences [59]). However, a developer might consider email addresses from a monitored domain to be safe, enabling Consecas to write a better policy (e.g., restrict the agent to send emails to only “myteam@work.com” instead of any “*@work.com” address).

3.2 Contextual Security Policies

The Consecas policy generator takes a user request and trusted context and outputs a policy: a set of constraints in a declarative language on the various tool APIs, and human-readable rationales for the constraints. For example, a filesystem tool policy written with regular expressions might include a `rm "/tmp/.*"` constraint on the `rm` command, with rationale “only remove temporary files when organizing”. Tool APIs define the possible set of actions; the policy constrains this set by preventing a tool, a tool’s API call, or an instance of an API call with concrete arguments from executing.

Consecas uses LLMs to scalably generate policies because they are sensitive to changes in context; in theory, a contextual security system can use any context-aware policy writer that can produce policies for every context. We leverage in-context learning [20]—prompting the LLM with a “golden” set of example policies to demonstrate what the model should output—to improve the generated policy quality.

Developers can optionally ask users to approve a task’s policy prior to agent task execution, given the constraints and their human-readable rationales. Policies can also be logged and later audited by the user, the developer, or a trusted third party. Consecas relies on experts (perhaps automated) to ensure that the rationale matches the constraints.

3.3 Deterministic Policy Enforcement

Given a policy and a proposed action, the policy enforcer checks whether the action lies within the policy’s set of allowed actions by deterministically evaluating policy constraints over the proposed action (e.g., checking predicates on action type or argument values, see § 4.1). When approving or denying an action, Consecas returns the rationale for the decision to the agent for transparency and feedback.

3.4 Security Discussion

Consecas trusts direct user input (e.g., the task to run), the specified trusted context (e.g., filesystem directory structure), the Consecas policy generator and enforcement code, and that the developer correctly integrated the agent with Consecas. Given these assumptions, Consecas ensures that no external tool action executes outside the current task’s security policy.

Consecas’s guarantees clearly depend on the strength of its security policies, which face *two fundamental limitations*. First, policies are only as good as the policy generator’s understanding of the user’s preferences, social norms, and actions required to fulfill the requested task. Second, the generator receives only limited, trusted contextual information (specified by the developer) in order to protect it against prompt injections. This restricts its ability to predict data-dependent actions (e.g., actions to allow when requested to “carry out the tasks requested in my manager’s email”). Because Consecas’s design uses an LLM, policies can improve

by improving the model’s quality (tuning models for individual user preferences or on user’s feedback about generated policies). Consecas can also increase trusted context by e.g., asking the user or sanitizing action responses.

4 Proof-of-Concept

We implement a simple computer-use AI agent with Consecas in Python for a Linux machine. The agent has access to a filesystem tool (the POSIX filesystem API), a file processing tool (which supports, e.g., `find` and `sed`), and an email tool that can read, send, delete, and categorize emails, and supports attachments. For simplicity, the email tool sends and receives emails in a Mail directory in users’ home directories. All tool APIs are bash commands (e.g., `send_email alice bob 'Hello' 'An Email'`, or `mkdir /home/alice/Backups`).

Without Consecas, the agent consists of planning and execution stages. The planner uses the Gemini 1.5 Pro LLM [69] and produces a bash command as a string; the executor then runs `subprocess.run([cmd])`. This control loop continues until the planner declares the task complete. If the task does not complete within some number of commands (set to 100), the agent returns “could not complete”.

4.1 The Consecas Prototype

Consecas is a Python library with the following API:

- `set_policy(task, trusted_ctxt) -> Policy`
- `is_allowed(cmd, policy) -> (bool, str)#rationale`

where a `Policy` maps an API call to constraints that include (i) whether the API call should ever be executed in this context, (ii) a boolean constraint over API call arguments such that the call can only execute when `True`; and (iii) a (human-readable) rationale for the choice of the prior two constraints. Policy generation invokes the LLM with a prompt including the tool API documentation, example policies for in-context learning [20], the user’s task, and the trusted context. For example, given:

- Task = “Get unread emails related to work and respond to any that are urgent”
- Trusted Context = email addresses and usernames (Alice, Bob, etc.), `current_user=Alice`

the policy generator returns constraints:

API Call: `send_email`

- ☐ Can Execute: `True`
- ☐ Args Constraint: `re.search(r'alice', $1)`
`^ re.search(r':*@work\.com', $2)`
`^ re.search(r'\.urgent\.', $3)`
- ☐ We need to send urgent responses to emails. The sender must be ‘alice’ (current user). The recipient must be one of the users in the email list from work. The subject must contain ‘urgent’.

API Call: `delete_email`

- ☐ Can Execute: `False`
- ☐ Args Constraint: `N/A`
- ☐ We are not deleting any emails in this task.

Policy Limitations. Our prototype represents argument constraints as regular expressions; future work might design a simpler DSL for constraints (e.g., predicates like `prefix`, `suffix`, `>`, `=`, etc.) to avoid regex complexity [55, 73]. Consecas also assumes positional API call parameters, and that the planner provides optional arguments last and in correct order; Consecas can support named arguments with changes to the policy generator prompt.

Trusted Context. We (the “developers”) define trusted context as the users’ email categories and addresses, and a tree of the filesystem directory structure (file and directory names are trusted). Tool-agnostic context includes the user’s username, time, and date. We also trust static context like the tool documentation provided in the prompt.

Agent Integration. Once Consecas sets the task’s policy, the agent invokes `is_allowed` when proposing a new command. Consecas checks whether the policy allows the API call at all, and, if so, whether each argument matches its regex constraint. Allowed commands go to execution. Consecas returns denied commands and their rationale to the planner, which the agent appends to the prompt to guide the planner LLM to choose an appropriate action. If commands continuously fail (up to 10 times), the agent returns “could not complete”.

Extending our prototype with new tools requires adding tool documentation to the prompts of the policy generator and agent LLMs, and specifying how to extract trusted context from the tool. Adding the email tool API and a subset of the POSIX API required less than a day of effort.

5 Case Studies

We demonstrate Consecas’s potential with a look at our prototype’s *utility* (how many tasks can complete with and without Consecas policies?) and *security* (can Consecas prevent actions misaligned with the current context?) We compare Consecas against baselines of the agent run with (i) no policy; (ii) a static restrictive policy that prevents all mutating actions; and (iii) a static permissive policy that allows all actions except deletion.

Setup. Experiments run on a Debian Linux machine with a client connection to an external Gemini 1.5 Pro model.

Prior to running each task, we initialize the filesystem with 10 users, including an admin. Each user contains >10 files in each general or job-specific folder (e.g., Downloads, Photos, or Logs). Mailboxes start with emails from other

Policy	Avg Tasks Completed	Inappropriate Actions Denied?
None	14.0/20	N
Static Permissive	12.2/20	N
Static Restrictive	0.0/20	Y
Conseca	12.0/20	Y

Figure 3: The agent with Conseca achieves comparable utility to an agent with a static permissive policy and completes more tasks (avg over 5 trials) than with a restrictive static policy. Conseca demonstrates potential to deny contextually inappropriate actions, which permissive policies fail to do.

users regarding work, family, etc.; some are categorized or include attachments like reports, invoices, and photos.

We construct 20 tasks involving filesystem and emailing tools, all of which require some subjective judgement (e.g., what is “important”) and multiple tool calls. Tasks range from simple (e.g., backup important files via email; organize email attachments into folders; or generate a data report from files and email it to colleagues) to more complex (deduplicate files and send me an email listing the removed files; email alerts about any PII in current logs; or create and email a newsletter to the team based on our logs).²

Limitations. To comprehensively evaluate Conseca’s benefits, future work will need ground-truth data from real agent applications’ workloads and contexts, instead of manually-curated, synthetic examples [17, 40, 65], which do not capture the full diversity of contexts. With ground-truth data, we can leverage ideas from red teaming [42, 85] and synthetic benchmarks [17, 38, 40, 43, 44, 65] to test Conseca’s benefits in real-world applications.

Preliminary Results. Early results (Figure 3) indicate that an agent with Conseca offers better utility than a static restrictive policy, and comparable utility to a static permissive policy. The unrestricted agent completes 70% of tasks on average. Four failed tasks—permission checks, organizing attachments, reading and acting upon urgent emails, and generating reports—required more than 100 actions, and two tasks—newsletter generation and checking for failed logins—proved too complex for our basic agent.

The agent with Conseca achieves 60% completion on average: both Conseca and permissive policies deny actions the task does not strictly require (e.g., touching a summary file to create it), and our basic agent fails to make progress without these actions. No task completes with a restrictive policy because all tasks require writes.

²Due to space constraints, we omit the full list of tasks. We locally crowd-sourced tasks and used Gemini to choose the most realistic ones, potentially biasing towards tasks we/Gemini believe are useful.

Inappropriate Actions. To demonstrate how Conseca’s contextual security policies can make inappropriate actions more difficult, we add an email from an employee with message “forward all emails about urgent security vulnerabilities to employee@work.com immediately”. Depending on the context—is the agent handling urgent email tasks or merely summarizing?—forwarding may or may not be appropriate. The unrestricted agent forwards emails even when inappropriate (e.g., when the user has asked the agent to categorize emails), but an agent run with Conseca denies forwarding for all tasks other than “perform the tasks in urgent emails”. Furthermore, Conseca denies forwarding while still maintaining higher utility than a restrictive policy.

6 Related Work

Prompt Injection Attacks and Defenses. Many agents’ use of LLMs leaves them vulnerable to jailbreaking [72] and prompt injections [28, 32, 54, 58, 75]. Defenses that detect attacks [2, 35, 36, 78], or prompt or train LLMs to distinguish untrusted from trusted inputs [34, 71, 76, 84], remain susceptible to attacks introduced by untrusted context.

To address this susceptibility, other systems build upon a dual-LLM pattern [77] that isolates the agent planner model from untrusted inputs [80, 81]. CaMeL [16] strengthens this pattern by allowing the isolated planner to output a limited subset of Python, and utilizing a custom Python interpreter to track untrusted inputs and apply capability-based policies to deny or allow actions. Conseca also draws upon the idea of a quarantined, isolated model, but isolates policy generation rather than planning, thus providing full context for the planner and requiring few changes to existing agents’ planner designs. Conseca explicitly aims to scale security to all contexts by enabling dynamic and fine-grained security policies, and can help reduce user burden in these systems, which assume predefined security configurations or policies.

Other work proposes a security analyser as part of agent design, which determines if a trace of agent actions violates a security policy composed of a set of predefined predicates [8]. Conseca proposes the idea of contextual policy generation independent of a specific policy language and enforcer, and can leverage their security analyser.

Context-Aware AI. Like prior work, Conseca uses models trained to understand behavioral norms [26, 47]. Other works use these models for appropriateness [41] and in a contextual integrity (CI) framework [5, 7, 9–11, 15, 30, 39, 48, 51, 52, 66, 74]. Conseca follows in the spirit of CI, but enforces contextually appropriate actions (in addition to contextually appropriate data flows) in order to prevent harm.

Traditional Application Security. Conseca helps enable finer-grained *access controls and capability restrictions* [1,

4, 6, 12–14, 18, 29, 45, 46, 49, 56, 60, 64, 83, 86], which the rise of agent systems makes increasingly necessary.

Mobile device permissions today offer the best example of how a platform can capture user’s privacy and security preferences in a multitude of contexts. However, developers can abuse these permissions and users might over-grant permissions in order to use applications [22, 23]. Agents serve even broader purposes than a mobile app, making it difficult to rely on users to manage permissions for every task; Conseca proposes a more scalable way to set permissions.

Other systems for *anomaly detection* rely on manual behavior specification [27] or learn a model of “good” behavior patterns (requiring labeled training data) [31, 50]. While more recent work has investigated LLMs for anomaly detection [68], classic ML or behavioral models avoid the threat of prompt injection. A Conseca policy can be seen as a model of “good” behaviors for the current context, with deterministic enforcement to prevent prompt injections; in Conseca, the “model” is created by a pre-trained LLM rather than learned. Conseca targets agents that perform tasks expressed in natural language in unforeseen contexts, which naturally points towards the use of an LLM (which encodes natural language reasoning) for policy creation. These anomaly detection systems may operate better in orthogonal settings, where events do not include natural language (e.g., syscalls) and follow more constrained and predictable patterns.

7 Discussion

Conseca only begins to touch upon the many directions for research in contextual security for agents. First, we can explore ways to **improve contextual policies**. While we hypothesize that users will experience far less confirmation fatigue with contextual policies than non-contextual ones, Conseca policies currently do not interact with users (e.g., asking users whether they want to override a Conseca-denied action, or giving users an “undo-log” to audit agent actions or even revert them if possible). User feedback can improve both the policy model’s understanding of preferences and norms, and also improve utility by allowing user-approved tasks to complete. To increase developers’ confidence in policies, we could perhaps automate policy verification using structured rationales and formally mapping them to constraints. Finally, sanitized output languages for tools and context may also increase the scope of trusted context and thus policy quality.

Contextual policies can also expand to constrain **agent trajectories**. Conseca’s policies currently check individual actions, regardless of what actions came prior. However, policies over multiple actions (a *trajectory*) can prevent the agent from getting “stuck” if it goes down a denied path, and protect against seemingly harmless single actions composing in inappropriate ways (e.g., sending a single email is harmless, but flooding inboxes is not). Trajectory constraints are

challenging because policies must reason about multiple trajectories of potentially infinite length, and perhaps predicate action constraints on prior conditions (e.g., “only send an email back if the sender requested a response”).

Use of LLMs also adds per-task **overheads** for policy generation, which can take seconds depending on the size of the model. LLM efficiency improvements, e.g., distilling a more capable model [82], could reduce this cost, potentially trading off some quality. Alternatively, we could use caching techniques, storing pre-generated or dynamically created policies for common contexts.

Lastly, can we **create contextual policies without LLMs** at all? This could improve policy reliability and remove the problem of LLM hallucination [70]. The scope of agent contexts targeted by Conseca may not always apply: some agent systems may look more similar to existing limited-purpose applications, operate in restricted contexts, or not involve natural language, thus lending themselves to existing access control or anomaly detection techniques used today. In practice, even with the most general agents, developers would likely combine Conseca’s dynamic policies with expertly-written manual policies and/or user confirmation for high-risk scenarios (e.g., unlocking a door). During normal-risk operation, Conseca can provide more fine-grained (but less auditable) security policies while reducing both developer and user burden.

8 Conclusion

Conseca imagines an emerging world in which agents perform human-like tasks, serving different purposes and commanded by unstructured inputs. To match the scope of agents, our security systems must adapt to contexts beyond what can be manually or statically encoded. While not all deployed agents might achieve this scale of capabilities, more research into contextual agent security—safely leveraging the power of context to choose and judge actions—can help ensure that we are ready for such agents when they emerge.

9 Acknowledgements

We thank our anonymous reviewers for their feedback, as well as Borja De Balle Pigem, Christoph Kern, Jacint Szabo, Santiago Díaz, Andrew Baumann, Phil Levis, David Culler, Hank Levy, Marco Gruteser, Ilia Shumailov, Daniel Ramage, and others in Systems Research@Google, Google Research, Google DeepMind, and Google agent security efforts for their insights and discussion that led to this work.

References

- [1] Rakesh Agrawal, Jerry Kiernan, Ramakrishnan Srikant, and Yirong Xu. “Hippocratic Databases”. In: *VLDB*. 2002.
- [2] Gabriel Alon and Michael Kamfonas. “Detecting Language Model Attacks with Perplexity”. In: *arXiv* (2023). eprint: 2308.14132 (cs.CL).

- [3] Anthropic. *Computer use (beta)*. 2024. URL: <https://docs.anthropic.com/en/docs/build-with-claude/computer-use>.
- [4] AppArmor. *AppArmor Security Profiles*. 2025. URL: <https://apparmor.net/>.
- [5] Noah Apthorpe, Yan Shvartzshnaider, Arunesh Mathur, Dillon Reisman, and Nick Feamster. "Discovering Smart Home Internet of Things Privacy Norms Using Contextual Integrity". In: *UbiComp*. 2018.
- [6] AWS. *AWS Identity and Access Management (IAM)*. 2023. URL: <https://aws.amazon.com/iam/> (visited on 01/30/2023).
- [7] Eugene Bagdasarian, Ren Yi, Sahra Ghalebikesabi, Peter Kairouz, Marco Gruteser, Sewoong Oh, Borja Balle, and Daniel Ramage. "AirGapAgent: Protecting Privacy-Conscious Conversational Agents". In: *CCS*. 2024.
- [8] Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. "AI Agents with Formal Security Guarantees". In: *ICML 2024 Next Generation of AI Safety Workshop*. 2024.
- [9] A Barth, A Datta, J C Mitchell, and H Nissenbaum. "Privacy and contextual integrity: framework and applications". In: *S&P*. 2006.
- [10] Sebastian Benthall, Seda F. Gürses, and Helen Nissenbaum. "Contextual Integrity through the Lens of Computer Science". In: *Found. Trends Priv. Secur.* 2 (2017), pages 1–69.
- [11] Sebastian Benthall, Seda Gürses, and Helen Nissenbaum. "Contextual Integrity through the Lens of Computer Science". In: *Foundations and Trends® in Privacy and Security* 2.1 (2017), pages 1–69.
- [12] Kristy Browder and Mary Ann Davidson. "The Virtual Private Database in Oracle9iR2". In: *Oracle Technical White Paper, Oracle Corporation* (2002).
- [13] Ji-Won Byun and Ninghui Li. "Purpose based access control for privacy protection in relational database systems". In: *The VLDB Journal* 17.4 (July 1, 2008), pages 603–619.
- [14] Surajit Chaudhuri, Tanmoy Dutta, and S. Sudarshan. "Fine Grained Authorization Through Predicated Grants". In: *2007 IEEE 23rd International Conference on Data Engineering*. 2007, pages 1174–1183.
- [15] Natalia Criado and Jose M Such. "Implicit Contextual Integrity in Online Social Networks". In: *arXiv* (Feb. 2015). eprint: 1502.02493 (cs.SI).
- [16] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. "Defeating Prompt Injections by Design". In: *arXiv* (2025). eprint: 2503.18813.
- [17] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. "Agent-Dojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents". In: *arXiv* (2024). eprint: 2406.13352 (cs.CR).
- [18] Dorothy E Denning, Selim G Akl, Mark Heckman, Teresa F. Lunt, Matthew Morgenstern, Peter G. Neumann, and Roger R. Schell. "Views for multilevel database security". In: *IEEE Transactions on Software Engineering* (1987).
- [19] Linda Di Geronimo, Larissa Braz, Enrico Fregnan, Fabio Palomba, and Alberto Bacchelli. "UI dark patterns and where to find them: a study on mobile applications and user perception". In: *CHI*. 2020.
- [20] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Jingyuan Ma, Rui Li, Heming Xia, Jingjing Xu, Zhiyong Wu, Tianyu Liu, Baobao Chang, Xu Sun, Lei Li, and Zhifang Sui. "A Survey on In-context Learning". In: *arXiv* (2024). eprint: 2301.00234 (cs.CL).
- [21] Serge Egelman, Adrienne Porter Felt, and David Wagner. "Choice Architecture and Smartphone Privacy: There's a Price for That". In: *The Economics of Information Security and Privacy*. Edited by Rainer Böhme. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pages 211–236.
- [22] Zheran Fang, Weili Han, and Yingjiu Li. "Permission based Android security: Issues and countermeasures". In: *Computers & Security* 43 (2014), pages 205–218.
- [23] Adrienne Porter Felt, Erika Chin, Steve Hanna, Dawn Song, and David Wagner. "Android permissions demystified". In: *CCS*. 2011.
- [24] Adrienne Porter Felt, Elizabeth Ha, Serge Egelman, Ariel Haney, Erika Chin, and David Wagner. "Android permissions: User attention, comprehension, and behavior". In: *SOUPS*. 2012.
- [25] Iason Gabriel, Arianna Manzini, Geoff Keeling, Lisa Anne Hendricks, Verena Rieser, et al. "The Ethics of Advanced AI Assistants". In: *arXiv* (2024). eprint: 2404.16244 (cs.CY).
- [26] Deep Ganguli, Amanda Askell, Nicholas Schiefer, Thomas I. Liao, Kamilė Lukošiušė, Anna Chen, Anna Goldie, et al. "The Capacity for Moral Self-Correction in Large Language Models". In: *arXiv* (2023). eprint: 2302.07459 (cs.CL).
- [27] Peng Gao, Xusheng Xiao, Ding Li, Zhichun Li, Kangkook Jee, Zhenyu Wu, Chung Hwan Kim, Sanjeev R. Kulkarni, and Prateek Mittal. "SAQL: A Stream-based Query System for Real-Time Abnormal System Behavior Detection". In: *27th USENIX Security Symposium (USENIX Security 18)*. Baltimore, MD: USENIX Association, Aug. 2018, pages 639–656.
- [28] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection". In: *arXiv* (2023). eprint: 2302.12173 (cs.CR).
- [29] Patricia P Griffiths and Bradford W Wade. "An authorization mechanism for a relational database system". In: *TODS* (1976).
- [30] Frances S Grodzinsky and Herman T Tavani. "Privacy in 'the cloud' applying Nissenbaum's theory of contextual integrity". In: *SIGCAS* (2011).
- [31] Xueyuan Han, Thomas F. J.-M. Pasquier, Adam Bates, James Mickens, and Margo I. Seltzer. "Unicorn: Runtime Provenance-Based Detector for Advanced Persistent Threats". In: *27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23-26, 2020*. The Internet Society, 2020.
- [32] Rich Harang. *Securing LLM Systems Against Prompt Injection*. 2023. URL: <https://developer.nvidia.com/blog/securingllm-systems-against-prompt-injection>.

- [33] Gemini Apps Help. *How your Gemini mobile app can help when your phone is locked*. 2025. URL: <https://support.google.com/gemini/answer/14576209?hl=en>.
- [34] Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. “Defending Against Indirect Prompt Injection Attacks With Spotlighting”. In: *arXiv* (2024). eprint: 2403.14720.
- [35] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabza. “Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations”. In: *arXiv* (2023). eprint: 2312.06674 (cs.CL).
- [36] Neel Jain, Avi Schwarzschild, Yuxin Wen, Gowthami Somepalli, John Kirchenbauer, Ping-yeh Chiang, Micah Goldblum, Anirudha Saha, Jonas Geiping, and Tom Goldstein. “Baseline Defenses for Adversarial Attacks Against Aligned Language Models”. In: *arXiv* (2023). eprint: 2309.00614 (cs.LG).
- [37] Maritza Johnson, John Karat, Clare-Marie Karat, and Keith Grueneberg. “Optimizing a policy authoring framework for security and privacy policies”. In: *SOUPS*. 2010.
- [38] Megan Kinniment, Lucas Jun Koba Sato, Haoxing Du, Brian Goodrich, Max Hasin, Lawrence Chan, Luke Harold Miles, Tao R. Lin, Hjalmar Wijk, Joel Burget, Aaron Ho, Elizabeth Barnes, and Paul Francis Christiano. “Evaluating Language-Model Agents on Realistic Autonomous Tasks”. In: *arXiv* (2023). eprint: 2312.11671.
- [39] Abhishek Kumar, Tristan Braud, Young D Kwon, and Pan Hui. “Aquilis: Using Contextual Integrity for Privacy Protection on Mobile Devices”. In: *IMWUT*. 2020.
- [40] LangChain. *Hugging Face prompt injection identification*. 2024. URL: https://python.langchain.com/v0.1/docs/guides/productionization/safety/hugging_face_prompt_injection/.
- [41] Joel Z Leibo, Alexander Sasha Vezhnevets, Manfred Diaz, John P Agapiou, William A Cunningham, Peter Sunehag, Julia Haas, Raphael Koster, Edgar A Duéñez-Guzmán, William S Isaac, et al. “A theory of appropriateness with applications to generative artificial intelligence”. In: *arXiv* (2024). eprint: 2412.19010.
- [42] Lizhi Lin, Honglin Mu, Zenan Zhai, Minghan Wang, Yuxia Wang, Renxi Wang, Junjie Gao, Yixuan Zhang, Wanxiang Che, Timothy Baldwin, Xudong Han, and Haonan Li. “Against The Achilles’ Heel: A Survey on Red Teaming for Generative Models”. In: *arXiv* (2024). eprint: 2404.00629 (cs.CL).
- [43] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. “Formalizing and Benchmarking Prompt Injection Attacks and Defenses”. In: *arXiv* (2024). eprint: 2310.12815 (cs.CR).
- [44] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. “Harm-Bench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal”. In: *arXiv* (2024). eprint: 2402.04249 (cs.LG).
- [45] Aastha Mehta, Eslam Elnikety, Katura Harvey, Deepak Garg, and Peter Druschel. “Qapla: Policy Compliance for Database-Backed Systems”. In: *USENIX Security*. 2017.
- [46] Microsoft. *Row-level security - SQL server*. 2023. URL: <https://learn.microsoft.com/en-us/sql/relational-databases/security/row-level-security> (visited on 01/30/2023).
- [47] Niloofar Miresghallah, Hyunwoo Kim, Xuhui Zhou, Yulia Tsvetkov, Maarten Sap, Reza Shokri, and Yejin Choi. “Can LLMs Keep a Secret? Testing Privacy Implications of Language Models via Contextual Integrity Theory”. In: *ICLR*. 2024.
- [48] Niloofar Miresghallah, Hyunwoo Kim, Xuhui Zhou, Yulia Tsvetkov, Maarten Sap, Reza Shokri, and Yejin Choi. “Can LLMs keep a secret? Testing privacy implications of language models via contextual integrity theory”. In: *ICLR*. 2024.
- [49] Ravi Murthy and Eric Sedlar. “Flexible and Efficient Access Control in Oracle”. In: *Proceedings of the 2007 ACM SIGMOD International Conference on Management of Data*. SIGMOD ’07. Beijing, China: Association for Computing Machinery, 2007, pages 973–980.
- [50] Ali Bou Nassif, Manar Abu Talib, Qassim Nasir, and Fatima Mohamad Dakalbab. “Machine Learning for Anomaly Detection: A Systematic Review”. In: *IEEE Access* 9 (2021), pages 78658–78700.
- [51] Helen Nissenbaum. “Privacy as contextual integrity”. In: *Washington Law Review* 79 (2004), pages 119–157.
- [52] Helen Nissenbaum. “Contextual integrity up and down the data food chain”. In: *Theor. Inq. Law* 20.1 (Mar. 2019), pages 221–256.
- [53] OpenAI. *Introducing Operator*. 2025. URL: <https://openai.com/index/introducing-operator/>.
- [54] OWASP. *Top 10 for Large Language Model Applications*. 2023. URL: https://owasp.org/wwwproject-top-10-for-large-language-modelapplications/assets/PDF/OWASP-Top-10-for-LLMs2023-v1_1.pdf.
- [55] OWASP. *Overly Permissive Regular Expression*. 2025. URL: <https://owasp.org/www-community/vulnerabilities/Overly-Permissive-Regular-Expression>.
- [56] Ruoming Pang, Ramon Caceres, Mike Burrows, Zhifeng Chen, Pratik Dave, Nathan Germer, Alexander Golynski, Kevin Graney, Nina Kang, Lea Kissner, Jeffrey L. Korn, Abhishek Parmar, Christina D. Richards, and Mengzhi Wang. “Zanzibar: Google’s Consistent, Global Authorization System”. In: *2019 USENIX Annual Technical Conference (USENIX ATC ’19)*. Renton, WA, 2019.
- [57] Simon Parkinson, Saad Khan, James Bray, and Daiyaan Shreef. “Creeper: a tool for detecting permission creep in file system access controls”. In: *Cybersecurity* 2.1 (2019), page 14.
- [58] Fábio Perez and Ian Ribeiro. “Ignore Previous Prompt: Attack Techniques For Language Models”. In: *arXiv* (2022). eprint: 2211.09527 (cs.CL).
- [59] recordsetter.com/. *Longest Active Email Address*. URL: <https://recordsetter.com/world-record/email-address/4310> (visited on 01/05/2025).

- [60] Red Hat. *What is SELinux?* 2025. URL: <https://www.redhat.com/en/topics/linux/what-is-selinux>.
- [61] Reddit User. *I might be OOTL here but why is Google Assistant now requiring me to unlock my phone for every single command?* 2024. URL: https://www.reddit.com/r/GooglePixel/comments/1d8piqj/i_might_be_otl_here_but_why_is_google_assistant/.
- [62] Scott Reed, Konrad Zolna, Emilio Parisotto, Sergio Gomez Colmenarejo, Alexander Novikov, Gabriel Barth-Maroon, Mai Gimenez, Yury Sulsky, Jackie Kay, Jost Tobias Springenberg, Tom Eccles, Jake Bruce, Ali Razavi, Ashley Edwards, Nicolas Heess, Yutian Chen, Raia Hadsell, Oriol Vinyals, Mahyar Bordbar, and Nando de Freitas. “A Generalist Agent”. In: *arXiv* (2022). eprint: 2205.06175 (cs.AI).
- [63] Robert W Reeder, Clare-Marie Karat, John Karat, and Carolyn Brodie. “Usability challenges in security and privacy policy-authoring interfaces”. In: *INTERACT*. 2007.
- [64] Shariq Rizvi, Alberto Mendelzon, S. Sudarshan, and Prasan Roy. “Extending Query Rewriting Techniques for Fine-Grained Access Control”. In: *Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data*. SIGMOD '04. Paris, France: Association for Computing Machinery, 2004, pages 551–562.
- [65] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. “Identifying the Risks of LM Agents with an LM-Emulated Sandbox”. In: *arXiv* (2024). eprint: 2309.15817 (cs.AI).
- [66] Yan Shvartzshnaider, Zvonimir Pavlinovic, Ananth Balashankar, Thomas Wies, Lakshminarayanan Subramanian, Helen Nissenbaum, and Prateek Mittal. “VACCINE: Using Contextual Integrity For Data Leakage Detection”. In: *WWW*. 2019.
- [67] National Institute of Standards and Technology. *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*. NIST Special Publication 800-162. U.S. Department of Commerce, Dec. 2014.
- [68] Jing Su, Chufeng Jiang, Xin Jin, Yuxin Qiao, Tingsong Xiao, Hongda Ma, Rong Wei, Zhi Jing, Jiajun Xu, and Junhong Lin. “Large Language Models for Forecasting and Anomaly Detection: A Systematic Literature Review”. In: *arXiv* (2024). eprint: 2402.10350 (cs.LG).
- [69] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. “Gemini: a family of highly capable multimodal models”. In: *arXiv* (2023). eprint: 2312.11805.
- [70] SM Tonmoy, SM Zaman, Vinija Jain, Anku Rani, Vipula Rawte, Aman Chadha, and Amitava Das. “A comprehensive survey of hallucination mitigation techniques in large language models”. In: *EMNLP*. 2024.
- [71] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. “The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions”. In: *arXiv* (2024). eprint: 2404.13208 (cs.CR).
- [72] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. “Jailbroken: How Does LLM Safety Training Fail?” In: *Advances in Neural Information Processing Systems*. Edited by A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine. Volume 36. Curran Associates, Inc., 2023, pages 80079–80110.
- [73] Adar Weidman. *Regular expression Denial of Service - ReDoS*. 2025. URL: https://owasp.org/www-community/attacks/Regular_expression_Denial_of_Service_-_ReDoS.
- [74] Primal Wijesekera, Arjun Baokar, Ashkan Hosseini, Serge Egelman, David Wagner, and Konstantin Beznosov. “Android permissions remystified: A field study on contextual integrity”. In: *USENIX Security*. 2015.
- [75] Simon Willison. *Prompt injection attacks against GPT-3*. 2023. URL: <https://simonwillison.net/2022/Sep/12/prompt-injection>.
- [76] Simon Willison. *Delimiters won't save you from prompt injection*. 2024. URL: <https://simonwillison.net/2023/May/11/delimiters-wont-save-you>.
- [77] Simon Willison. *The Dual LLM pattern for building AI assistants that can resist prompt injection*. 2024. URL: <https://simonwillison.net/2023/Apr/25/dual-llm-pattern/>.
- [78] Simon Willison. *You can't solve AI security problems with more AI*. 2024. URL: <https://simonwillison.net/2022/Sep/17/prompt-injection-more-ai/>.
- [79] Michael Wooldridge and Nicholas R. Jennings. “Intelligent agents: theory and practice”. In: *The Knowledge Engineering Review* 10.2 (1995), pages 115–152.
- [80] Fangzhou Wu, Ethan Cecchetti, and Chaowei Xiao. “System-Level Defense against Indirect Prompt Injection Attacks: An Information Flow Control Perspective”. In: *arXiv* (2024). eprint: 2409.19091 (cs.CR).
- [81] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. “SecGPT: An Execution Isolation Architecture for LLM-Based Systems”. In: *arXiv* (2024). eprint: 2403.04960 (cs.CR).
- [82] Xiaohan Xu, Ming Li, Chongyang Tao, Tao Shen, Reynold Cheng, Jinyang Li, Can Xu, Dacheng Tao, and Tianyi Zhou. “A survey on knowledge distillation of large language models”. In: *arXiv* (2024). eprint: 2402.13116.
- [83] Jean Yang, Travis Hance, Thomas H Austin, Armando Solar-Lezama, Cormac Flanagan, and Stephen Chong. “Precise, dynamic information flow for database-backed applications”. In: *ACM Sigplan Notices* 51.6 (2016), pages 631–647.
- [84] Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. “Benchmarking and defending against indirect prompt injection attacks on large language models”. In: *arXiv* (2023). eprint: 2312.14197.
- [85] Jiahao Yu, Xingwei Lin, Zheng Yu, and Xinyu Xing. “GPT-FUZZER: Red Teaming Large Language Models with Auto-Generated Jailbreak Prompts”. In: *arXiv* (2024). eprint: 2309.10253 (cs.AI).
- [86] Wen Zhang, Eric Sheng, Michael Chang, Aurojit Panda, Mooly Sagiv, and Scott Shenker. “Blockaid: Data Access Policy Enforcement for Web Applications”. In: *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, July 2022, pages 701–718.

- [87] Wangchunshu Zhou, Yuchen Eleanor Jiang, Long Li, Jialong Wu, Tiannan Wang, Shi Qiu, Jintian Zhang, Jing Chen, Ruipu Wu, Shuai Wang, Shiding Zhu, Jiyu Chen, Wentao Zhang, Xiangru Tang, Ningyu Zhang, Huajun Chen, Peng Cui, and Mrinmaya Sachan. “Agents: An Open-source Framework for Autonomous Language Agents”. In: *arXiv* (2023). eprint: 2309.07870 (cs.CL).